

# Algorithms and data structures I.

Hash tables

# New data structure: Hash tables

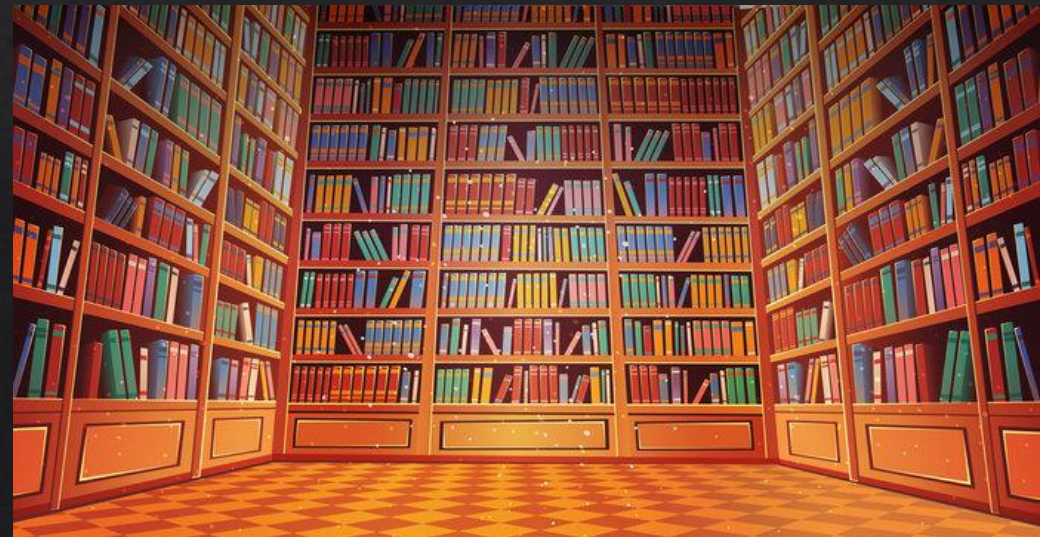
- ◆ Goal: efficient SEARCH on a set of known (unique) keys,  
with also efficient DELETION and INSERTION methods
  
- ◆ We already know:
  1. Unordered list/array + linear search  $O(n)$
  2. Ordered array + logarithmic (binary) search  $O(\log n)$
  3. Ordered list + Efficient linear search  $O(n)$
  4. Binary searchtree:  $O(h)$  where  $h$  is the height of the tree.
  
- ◆ Hash: Possible to even do statistically CONSTANT time.



# Illustration problem

- ◆ Let's assume you want to fill up a library with books and not just stuff them in there, but you want to be able to easily find them again when you need them.
- ◆ With the title of the book, can give you the right spot at once, so all you have to do is just stroll over to the right shelf, and pick up the book.
- ◆ But how can that be done? Well, with a bit of forethought, instead of just starting to fill up the library from one end to the other, you devise a clever little method. You take the title of the book, run it through a small computer program, which spits out a shelf number and a slot number on that shelf. This is where you place the book.
- ◆ The beauty of this program is that later on, when a person comes back in to read the book, you feed the title through the program once more, and get back the same shelf number and slot number that you were originally given, and this is where the book is located.

Source: <https://stackoverflow.com/a/730813>



Source: [https://stock.adobe.com/search?k=library+cartoon&asset\\_id=266015225](https://stock.adobe.com/search?k=library+cartoon&asset_id=266015225)

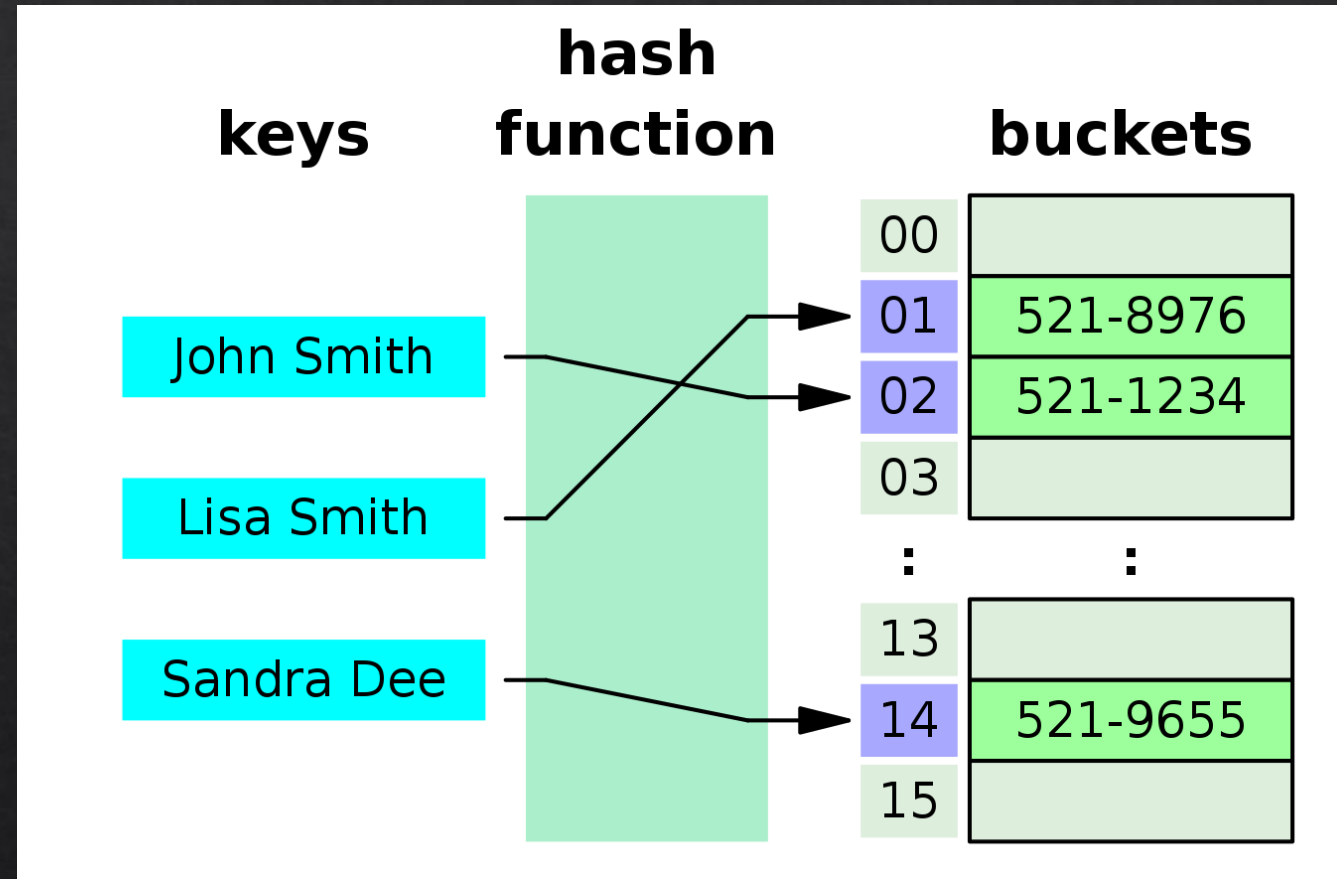
# I. Direct-addressing

# I. Direct-addressing

How does it work?

- ❖ Instead of inserting every new element one after the other as new neighbours
- ❖ Every element has a unique KEY (e.g. books with author + title)
- ❖ Convert the complicated KEY into a Data structure address (e.g. starting with index 0 until  $n$ ,  $n := \text{size}-1$ ) using the HASH function
- ❖ Put the element to this specific location.

NEW PROBLEM ?????

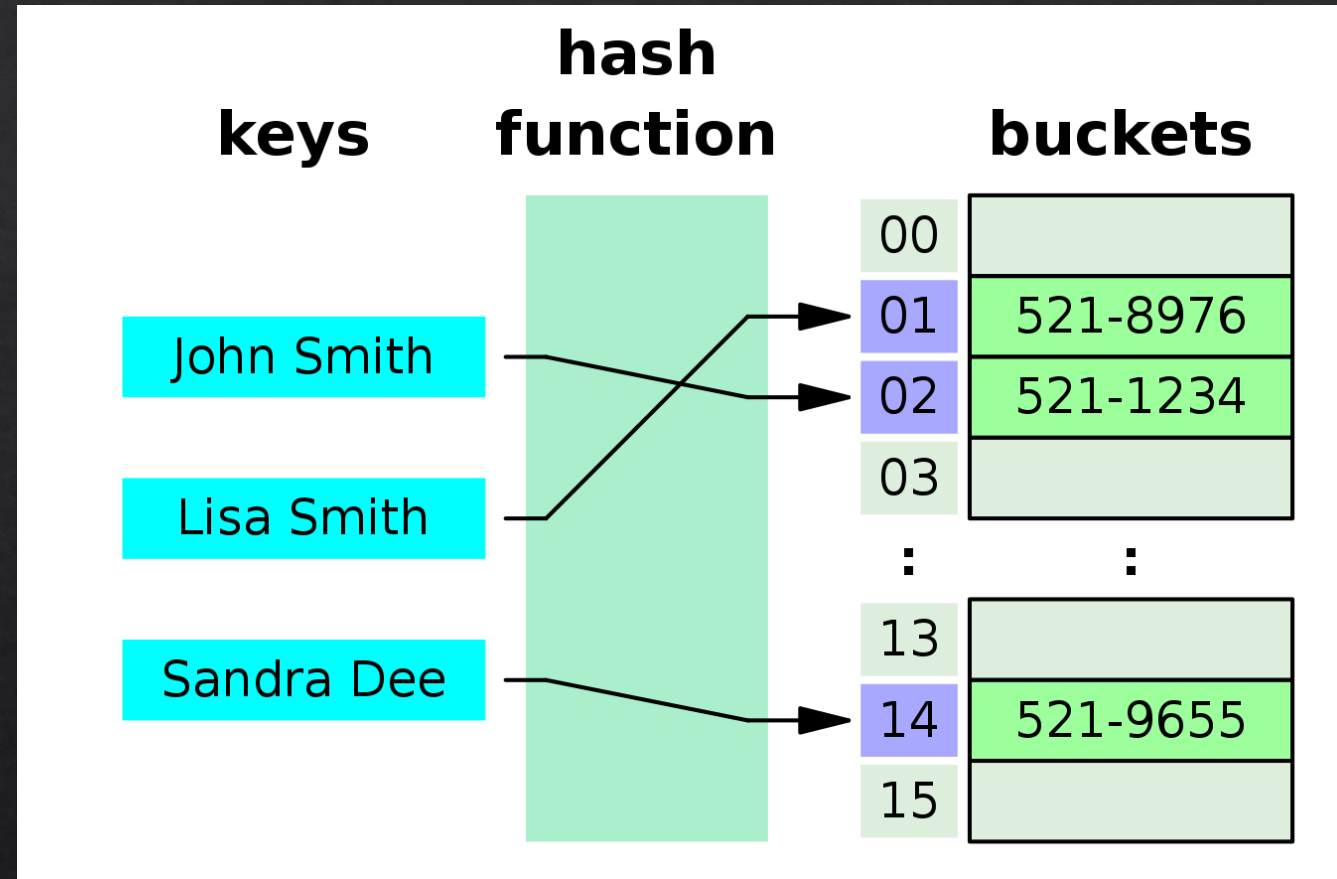




# I. Direct-addressing

What are the problems with this method?

- ◇ Space problem:
  - ◇ If I create a specific unique slot for every possible element
  - ◇ Many of wasted space if not many elements are stored from the KEY UNIVERSE  
(notation:  $|U| \gg n$ )
- ◇ Possible solution:
  - ◇ Create a smaller data structure than the KEY UNIVERSE



NEW PROBLEM??????

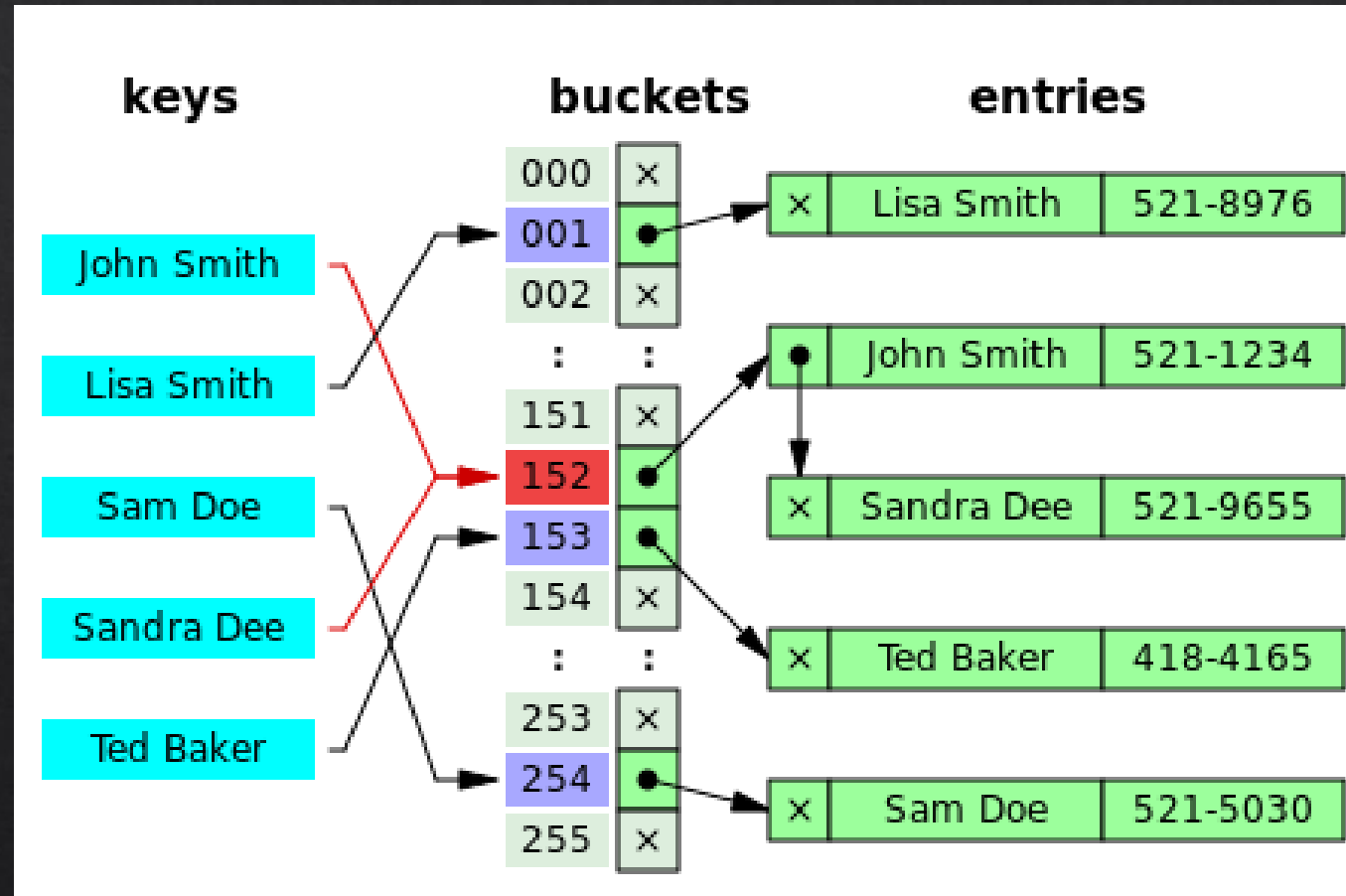
# I. Direct-addressing

What are the problems with this method?

- ◆ COLLISION problem:
  - ◆ Restricting the addresses smaller than the UNIVERSE will result in multiple keys having the same specific slot
- ◆ Possible solution:
  - ◆ Create a list at every address, and add the new elements into the list

ADVANTAGES ?????

DISADVANTAGE ?????



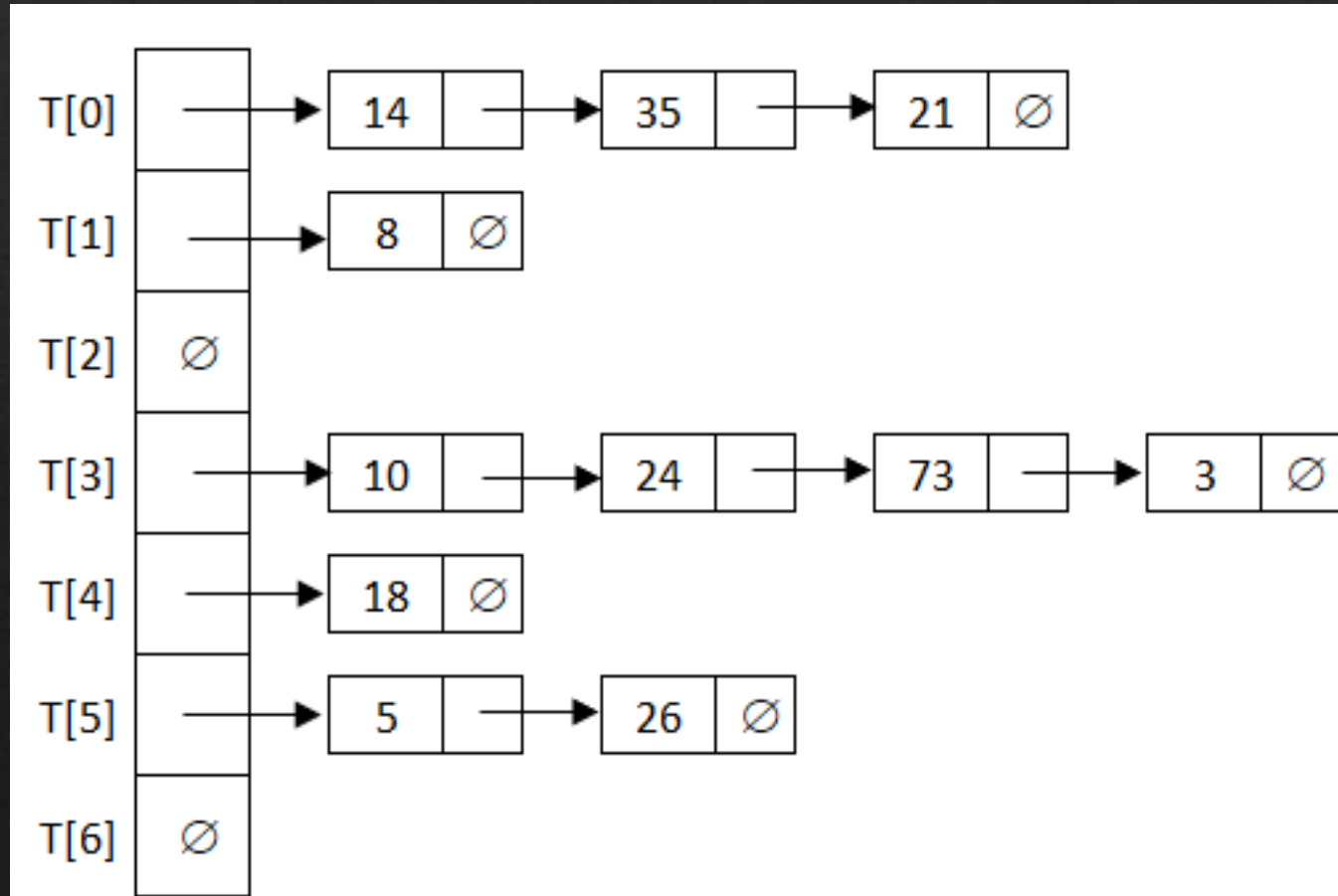
# I. Direct-addressing

## Task:

♦  $m = 7$ ,  $h : \mathbb{N} \rightarrow 0..6$ ,  $h(k) = k \bmod 7$     Insert: 3, 21, 73, 8, 24, 35, 26, 14, 5, 10, 18.

- ♦  $k$ : key of the element  
 $m$ : size of the table  
 $h(k)$ : hash function for the address

| k  | h(k) |
|----|------|
| 3  | 3    |
| 21 | 0    |
| 73 | 3    |
| 8  | 1    |
| 24 | 3    |
| 35 | 0    |
| 26 | 5    |
| 14 | 0    |
| 5  | 5    |
| 10 | 3    |
| 18 | 4    |





# Hash functions

# Hash functions

- ◆ The  $h : U \rightarrow [0..m)$  Function is good if it distributes the possible keys evenly in the hash table.
- ◆ **Division method:**
- ◆  $h(k) = k \bmod m$
- ◆ It's quality depends on  $m$ , and it's usually a good idea to make it a prime, which is far from some exponentials of 2.

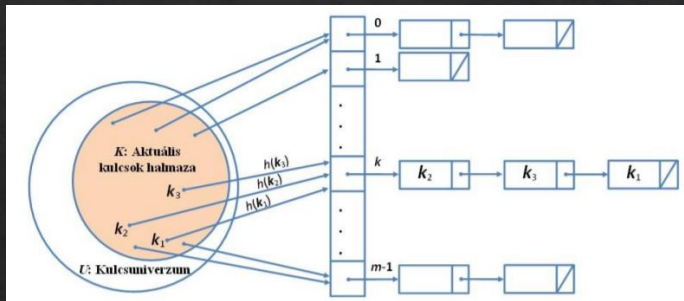
Load factor of the table:  $\alpha = n/m$  ( $n$  records in the table,  $m$  length of table). This gives a good approximation of how long the lists are in chained approaches, or how many key collisions it takes to successfully finish insertion.

# Hash functions

- ◆ **Multiplication method:**
- ◆ Let  $0 < A < 1$  be a constant (non-trivial values).
- ◆ Let's take the fraction part:  $\{k * A\} < 1$
- ◆  $h(k) = \lfloor \{k * A\} * m \rfloor$   $h(k) \in [0..m-1]$  true.
- ◆  $m$  can be picked freely
- ◆ Fun fact: D. E. Knuth advice for  $A$ :  $0.618\dots$  (1 - golden ratio)



# Chained solution



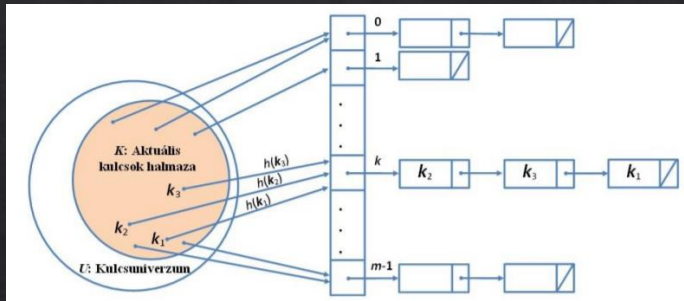
| $E1$                                    |
|-----------------------------------------|
| $+key : U$                              |
| $\dots$ // satellite data may come here |
| $+next : E1^*$                          |
| $+E1() \{ next := \oslash \}$           |

|                            |
|----------------------------|
| $\text{init}( T:E1^*[m] )$ |
| $i := 0 \text{ to } m-1$   |
| $T[i] := \oslash$          |

|                                        |
|----------------------------------------|
| $\text{search}( T:E1^*[] ; k:U ):E1^*$ |
| return $\text{searchS1L}(T[h(k)],k)$   |

|                                                  |
|--------------------------------------------------|
| $\text{searchS1L}( q:E1^* ; k:U ):E1^*$          |
| $q \neq \oslash \wedge q \rightarrow key \neq k$ |
| $q := q \rightarrow next$                        |
| return $q$                                       |

# Chained solution



| $\mathbf{E1}$                           |
|-----------------------------------------|
| $+key : U$                              |
| $\dots$ // satellite data may come here |
| $+next : \mathbf{E1}^*$                 |
| $+\mathbf{E1}() \{ next := \oslash \}$  |

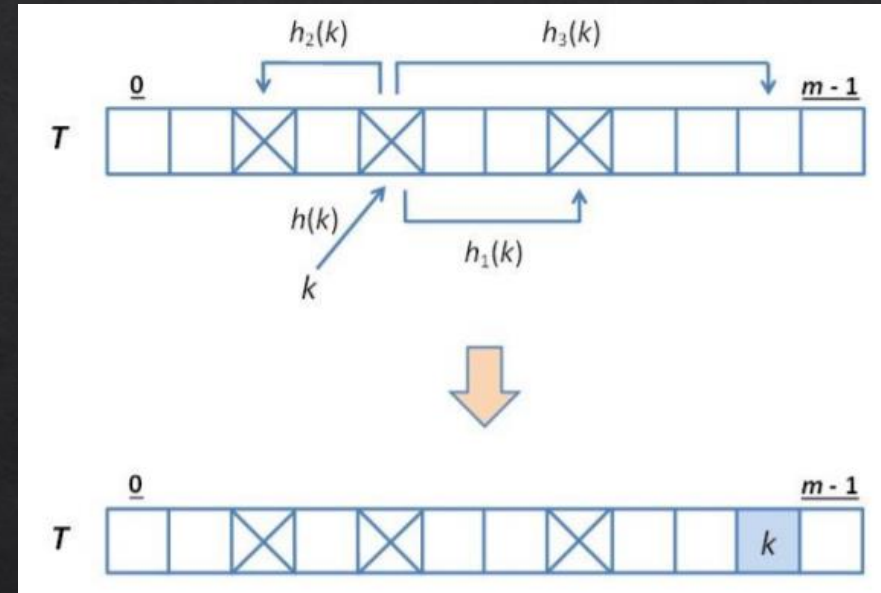
| $\text{searchS1L}(q:\mathbf{E1}^* ; k:U):\mathbf{E1}^*$ |
|---------------------------------------------------------|
| $q \neq \oslash \wedge q \rightarrow key \neq k$        |
| $q := q \rightarrow next$                               |
| return $q$                                              |

| $\text{insert}(T:\mathbf{E1}^*[] ; p:\mathbf{E1}^*):\mathbb{B}$ |
|-----------------------------------------------------------------|
| $k := p \rightarrow key ; s := h(k)$                            |
| $\text{searchS1L}(T[s], k) = \oslash$                           |
| $p \rightarrow next := T[s]$                                    |
| $T[s] := p$                                                     |
| return $true$                                                   |
| return $false$                                                  |

| $\text{remove}(T:\mathbf{E1}^*[] ; k:U):\mathbf{E1}^*$ |
|--------------------------------------------------------|
| $s := h(k) ; p := \oslash ; q := T[s]$                 |
| $q \neq \oslash \wedge q \rightarrow key \neq k$       |
| $p := q ; q := q \rightarrow next$                     |
| $q \neq \oslash$                                       |
| $p = \oslash$                                          |
| $T[s] := q \rightarrow next$                           |
| $p \rightarrow next := q \rightarrow next$             |
| $q \rightarrow next := \oslash$                        |
| return $q$                                             |
| SKIP                                                   |

## II. Open-addressing

- ◆ Each record goes into the table directly
- ◆ We resolve collisions by trying to find a new free slot somehow
- ◆ Potential probing sequence:
  - ◆  $h(k,0) \ h(k,1) \ h(k,2) \ \dots \ h(k,m-1)$
  - ◆ The probing sequence must 'touch' every slot once, therefore is it the permutation of  $0..m-1$  integers
- ◆ For the same key we must get the same probing sequence





# Notations

|                                                                                                                                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\begin{array}{l} \mathbf{R} \\ + k : U \cup \{E, D\} // k \text{ is a key or it is Empty or Deleted} \\ + \dots // \text{satellite data} \end{array}$ |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|

- ◇  $k$  is a key,  
E – empty, D- deleted. E,D are extremal values
- ◇  $h: U \times 0..(m-1) \rightarrow 0..(m-1)$  : hashing sequence  
 $\langle h(k,0), h(k,1), h(k,2), \dots, h(k,m-1) \rangle$  : potential probing sequence
- ◇ We say that an E, and D slots are 'free', otherwise occupied.
- ◇ Probing sequence:
  - ◇ **Linear, Quadratic, and double hash** (*last is well distributed*)

# Open-addressing and operations with deletions.

- ◇ First consider search and insertion only
- ◇ **Search:** We follow the potential probing sequence until we find E, D or k itself. Given that we find E, it means that the key is not part of the hash table, and therefore it's not successful. If we find a D, that is the same as if the slot were occupied, since k could occur later on.
- ◇ **Insertion:** We follow the potential probing sequence until we find E, D or k itself. Given that we find E, it means that the key is not part of the hash table, and therefore we can insert it right there. If we find a D, that is the same as if the slot were occupied, however we must remember this slot since the next time we find an E, k should be inserted into the fi



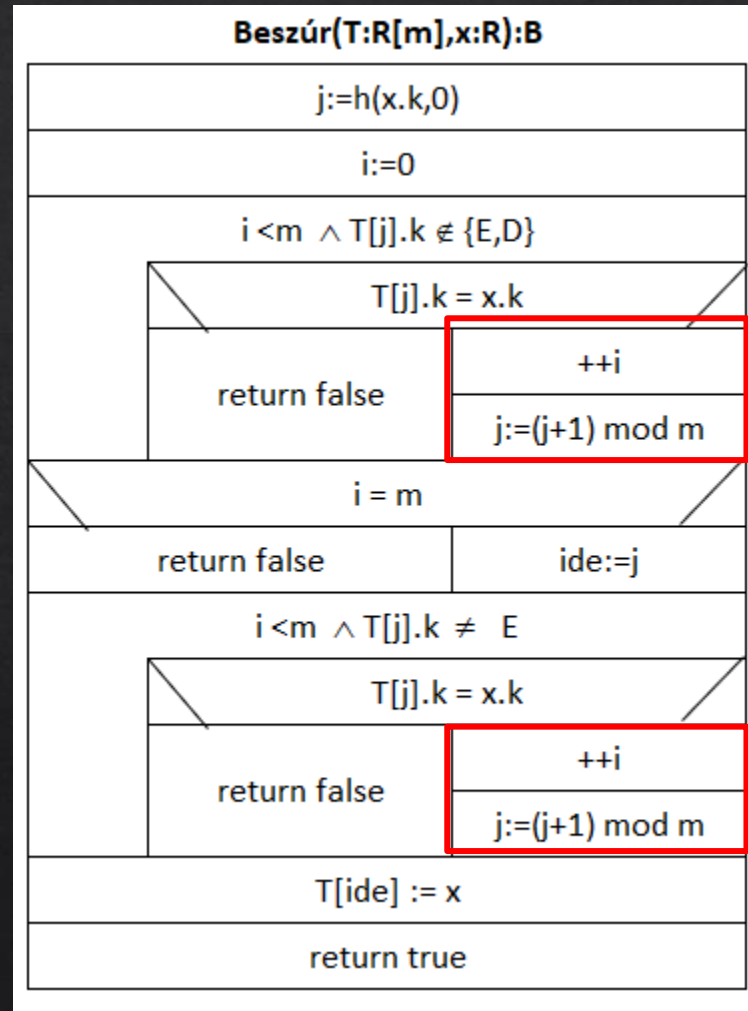
## II-1: Linear probing

Linear     $m = 11$      $N \rightarrow 0 \dots 10$      $h(k,n) = (k + n) \bmod 11$

| Operation | Key | $h(k)$ | Probe sequence         | Result? | 0  | 1 | 2  | 3  | 4  | 5  | 6  | 7 | 8 | 9 | 10 |
|-----------|-----|--------|------------------------|---------|----|---|----|----|----|----|----|---|---|---|----|
| Insert    | 24  | 2      | $2_E$                  | ✓       |    |   | 24 |    |    |    |    |   |   |   |    |
| Insert    | 16  | 5      | $5_E$                  | ✓       |    |   | 24 |    |    | 16 |    |   |   |   |    |
| Insert    | 57  | 2      | $2, 3_E$               | ✓       |    |   | 24 | 57 |    | 16 |    |   |   |   |    |
| Insert    | 32  | 10     | $10_E$                 | ✓       |    |   | 24 | 57 |    | 16 |    |   |   |   | 32 |
| Insert    | 15  | 4      | $4_E$                  | ✓       |    |   | 24 | 57 | 15 | 16 |    |   |   |   | 32 |
| Delete    | 57  | 2      | $2, 3_{57}$            | ✓       |    |   | 24 | D  | 15 | 16 |    |   |   |   | 32 |
| Insert    | 21  | 10     | $10, 0_E$              | ✓       | 21 |   | 24 | D  | 15 | 16 |    |   |   |   | 32 |
| Search    | 2   | 2      | $2, 3, 4, 5, 6_E$      | ×       | 21 |   | 24 | D  | 15 | 16 |    |   |   |   | 32 |
| Insert    | 2   | 2      | $2, 3_D, 4, 5, 6_E$    | ✓       | 21 |   | 24 | 2  | 15 | 16 |    |   |   |   | 32 |
| Insert    | 2   | 2      | $2, 3_2$               | ×       | 21 |   | 24 | 2  | 15 | 16 |    |   |   |   | 32 |
| Search    | 21  | 10     | $10, 0_{21}$           | ✓       | 21 |   | 24 | 2  | 15 | 16 |    |   |   |   | 32 |
| Insert    | 35  | 2      | $2, 3, 4, 5, 6_E$      | ✓       | 21 |   | 24 | 2  | 15 | 16 | 35 |   |   |   | 32 |
| Delete    | 15  | 4      | $4_{15}$               | ✓       | 21 |   | 24 | 2  | D  | 16 | 35 |   |   |   | 32 |
| Insert    | 35  | 2      | $2, 3, 4_D, 5, 6_{35}$ | X       | 21 |   | 24 | 2  | D  | 16 | 35 |   |   |   | 32 |



# Algorithm for insertion



## II-2: Quadratic probing

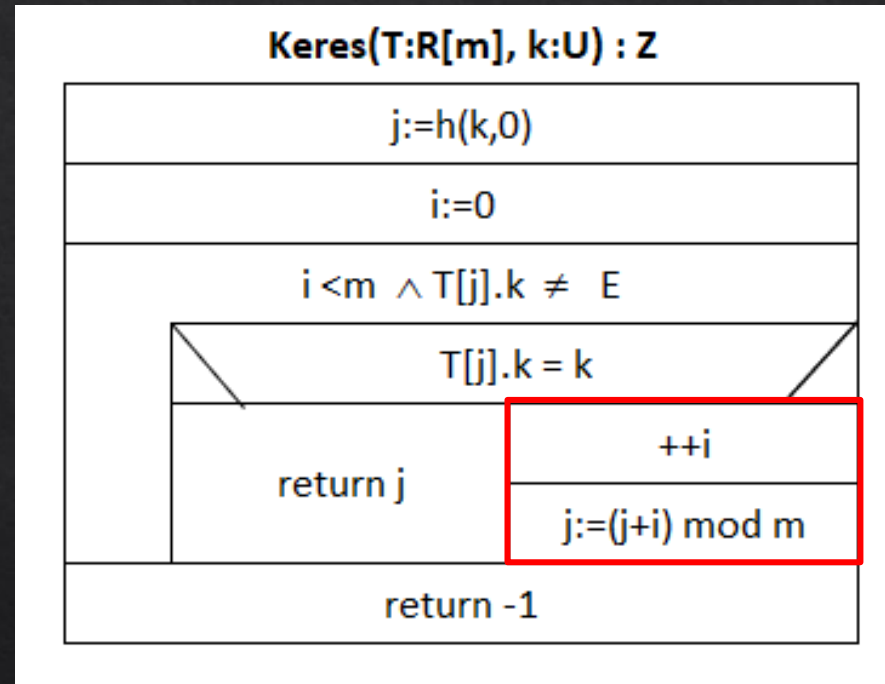
- ◇  $h(k,i) = (h(k,0) + c_1 * i + c_2 * i^2) \bmod m$
- ◇  $c_1$  and  $c_2$  constants ( $c_2 \neq 0$ ).
- ◇ Given that  $m$  is an exponential of 2,  $c_1=1/2$  and  $c_2=1/2$  are good solutions.
- ◇  $h(k,i) = (h(k,0) + \frac{1}{2} * i + \frac{1}{2} * i^2) \bmod m, i=0..m-1$ .
- ◇  $h(k,i) = (h(k,0) + 1 + 2 + 3 + \dots + i) \bmod m, i=0..m-1$ .

## II-2: Quadratic

Quadratic  $m = 8$   $h(k,n) = (k + (n + n^2)/2) \bmod 8$

| Operation | Key | $h_1(k)$ | Probe sequence                  | Result? | 0  | 1  | 2  | 3  | 4  | 5  | 6 | 7  |
|-----------|-----|----------|---------------------------------|---------|----|----|----|----|----|----|---|----|
| Insert    | 13  | 5        | $5_E$                           | ✓       |    |    |    |    |    | 13 |   |    |
| Insert    | 20  | 4        | $4_E$                           | ✓       |    |    |    |    | 20 | 13 |   |    |
| Insert    | 31  | 7        | $7_E$                           | ✓       |    |    |    |    | 20 | 13 |   | 31 |
| Insert    | 87  | 7        | $7, 0_E$                        | ✓       | 87 |    |    |    | 20 | 13 |   | 31 |
| Insert    | 12  | 4        | $4, 5, 7, 2_E$                  | ✓       | 87 |    | 12 |    | 20 | 13 |   | 31 |
| Delete    | 31  | 7        | $7_{31}$                        | ✓       | 87 |    | 12 |    | 20 | 13 |   | D  |
| Search    | 12  | 4        | $4, 5, 7, 2_{12}$               | ✓       | 87 |    | 12 |    | 20 | 13 |   | D  |
| Search    | 15  | 7        | $7, 0, 2, 5, 1_E$               | ×       | 87 |    | 12 |    | 20 | 13 |   | D  |
| Insert    | 4   | 4        | $4, 5, \underline{7}_D, 2, 6_E$ | ✓       | 87 |    | 12 |    | 20 | 13 |   | 4  |
| Delete    | 10  | 2        | $2, 3_E$                        | ×       | 87 |    | 12 |    | 20 | 13 |   | 4  |
| Insert    | 35  | 3        | $3_E$                           | ✓       | 87 |    | 12 | 35 | 20 | 13 |   | 4  |
| Insert    | 10  | 2        | $2, 3, 5, 0, 4, 1_E$            | ✓       | 87 | 10 | 12 | 35 | 20 | 13 |   | 4  |

# Search for quadratic





## II-3: Double hashing

- ◇ Similar to linear hashing, but instead of stepping one each time, the step size is determined by a second hash function
- ◇  $h_1: U \rightarrow 0..m-1$ , Provides the initial place of the key
- ◇  $h_2: U \rightarrow 1..m-1$ , Provides the step size
- ◇  $h(k,i) = (h_1(k) + i * h_2(k)) \bmod m$ ,  $i=0..m-1$  is the full hash function
- ◇  $h_2(k)$  properties:
  - ◇  $h_2(k) > 0$
  - ◇  $h_2(k)$  and  $m$  relative primes ( $m$  is usually prime)

## II-3: Double hashing

| Double    | m=  | 11       | $h_1(k) = k \bmod 11$ | $h_2(k) = 1 + (k \bmod 10)$ |         |    |    |     |   |   |   |    |   |   |    |    |
|-----------|-----|----------|-----------------------|-----------------------------|---------|----|----|-----|---|---|---|----|---|---|----|----|
| Operation | Key | $h_1(k)$ | $h_2(k)$              | Probe sequence              | Result? | 0  | 1  | 2   | 3 | 4 | 5 | 6  | 7 | 8 | 9  | 10 |
| Insert    | 23  | 1        | 4                     | $1_E$                       | ✓       |    | 23 |     |   |   |   |    |   |   |    |    |
| Insert    | 42  | 9        | 3                     | $9_E$                       | ✓       |    | 23 |     |   |   |   |    |   |   | 42 |    |
| Insert    | 31  | 9        | 2                     | $9, 0_E$                    | ✓       | 31 | 23 |     |   |   |   |    |   |   | 42 |    |
| Insert    | 110 | 0        | 1                     | $0, 1, 2_E$                 | ✓       | 31 | 23 | 110 |   |   |   |    |   |   | 42 |    |
| Insert    | 55  | 0        | 6                     | $0, 6_E$                    | ✓       | 31 | 23 | 110 |   |   |   | 55 |   |   | 42 |    |
| Delete    | 31  | 9        | 2                     | $9, 0_{31}$                 | ✓       | D  | 23 | 110 |   |   |   | 55 |   |   | 42 |    |
| Delete    | 55  | 0        | 6                     | $0, 6_{55}$                 | ✓       | D  | 23 | 110 |   |   |   | D  |   |   | 42 |    |
| Search    | 13  | 2        | 4                     | $2, 6, 10_E$                | ×       | D  | 23 | 110 |   |   |   | D  |   |   | 43 |    |
| Insert    | 6   | 6        | 7                     | $6_D, 2, 9, 5_E$            | ✓       | D  | 23 | 110 |   |   |   | 6  |   |   | 44 |    |
| Delete    | 31  | 9        | 2                     | $9, 0, 2, 4_E$              | ×       | D  | 23 | 110 |   |   |   | 6  |   |   | 44 |    |
| Insert    | 13  | 2        | 4                     | $2, 6, 10_E$                | ✓       | D  | 23 | 110 |   |   |   | 6  |   |   | 44 | 13 |
| Search    | 11  | 0        | 2                     | $0, 2, 4_E$                 | ×       | D  | 23 | 110 |   |   |   | 6  |   |   | 44 | 13 |

# Search for double hashing

